

ICLR 2024 | 表格数据也能有自己的“基础模型”

港大与北京智源、Creatify AI 提出 UniTabE：一套不挑表格结构的通用预训练协议

过去几年，预训练几乎重塑了自然语言处理和计算机视觉：先在海量无标注数据上学到通用知识，再迁移到各种下游任务。可当我们把目光转回数据科学最常打交道的表格数据，会发现它几乎被这浪潮落下了。股价预测、房价估计、信用评分、保险定价，背后都是一张张行列排布的表，却始终没有一个被广泛采用的“表格基础模型”。

问题的根子不在数据量，而在结构。每一张表的“模式（schema）”都不一样：列名不同、数据类型不同、列的数量也不同。一个在某张表上训练好的模型，往往没法直接搬到另一张表上用。这让“预训练一次、到处迁移”的范式在表格世界里很难落地。

这篇发表于 ICLR 2024 的工作 UniTabE 想把这条路走通。它提出一套与表结构无关（schema-agnostic）的通用预训练协议：用一个小模块统一表示任意一张表的每个单元格，用 Transformer 精炼整张表的表示，再借助自由文本提示（free-form prompt）和一个浅层自回归解码器适配不同任务。作者在从 Kaggle 采集的约 13B 条样本上完成预训练，最终在一系列基准上超过了长期作为工业标配的 XGBoost 以及此前的表格模型。

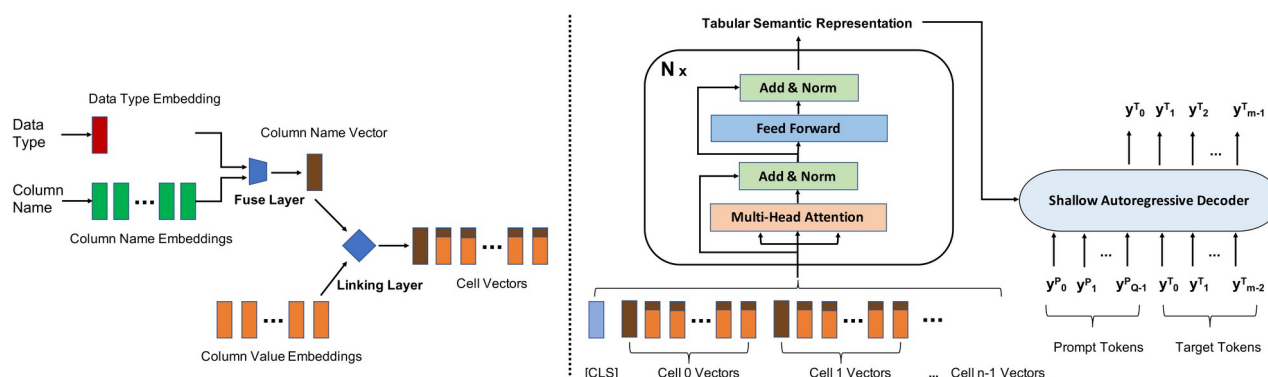


图 1 UniTabE 总体架构。左侧是 TabUnit 模块处理单个单元格的流程：数据类型嵌入经 Fuse Layer 融入列名向量，再由 Linking Layer 注入到每个数值 token，得到单元格向量。右侧是整体架构：所有单元格向量拼接后前缀一个 [CLS] token，送入 N 层 Transformer 编码器得到表格语义表示，最后由一个浅层自回归解码器在提示（Prompt）引导下逐 token 生成目标（Target）。

论文与代码：论文见 arXiv:2307.09249 (<https://arxiv.org/abs/2307.09249>)。

为什么表格数据这么难“预训练”

表格看起来简单，却恰恰因为“简单得太自由”而难办。文本和图像有天然统一的输入形式，一段话就是一串 token，一张图就是一个像素网格；而表格的列名、列数、每列的数据类型几乎每张表都不一样。模型如果把某张表的结构写死在架构里，换一张表就得重新设计。

此前的做法大体分两条路，各有代价。一条是把整张表“文本化”，拉平成一段文字喂给语言模型，这样结构统一了，但数值的数量含义被当成普通字符串丢掉了。另一条是要求训练和测试时的表结构必须一致，这在列会随时间增减的真实场景里很脆弱。UniTabE 想要的是第三条路：既保留单元格层面的结构信息，又不把任何固定结构写死。

核心思路：把每个单元格当成一个键值对

UniTabE 的核心是一个叫 TabUnit 的小模块，它一次只处理一个单元格，并把单元格看作“键值对”：列名是键，单元格内容是值。列名先被嵌入并均值池化成一个列名向量；与此同时，一个单独的数据类型嵌入（Data Type Embedding，标记该列是数值、类别还是文本）通过一个门控机制“融入”列名向量。这一步叫 Fuse Layer，好处是同一个“薪资”列，无论取值是数字还是 high/medium/low 这样的词，都能被一致地对待。

紧接着的 Linking Layer 把列名向量注入到该列每个取值 token 上，让后续 Transformer 的自注意力清楚地知道“哪个值属于哪一列”。所有单元格向量拼接起来，前缀一个 [CLS] 特殊 token，送入 N 层 Transformer 编码器，得到整张表的语义表示。解码端刻意设计得很浅，是一个浅层自回归解码器（论文中用 LSTM 或 Transformer 皆可）：它在 [CLS] 状态和一段自由文本提示（例如“fill in missing value, salary”）的引导下逐 token 生成答案。之所以让解码器“弱一点”，是为了迫使大部分知识沉淀在可复用的编码器里。

预训练混合了两种自监督信号。一是多单元格掩码（multi-cell-masking）：随机遮住若干整个单元格，让模型把它们的内容还原出来，用最大对数似然优化。二是对比学习（contrastive learning）：把来自同一样本、彼此重叠的行块（row block）拉近，把来自不同样本的行块推远。微调时，任何下游任务都被统一改写成“在特定提示下预测被遮住的目标列”，于是预训练与微调共用同一套框架。

撑起预训练的：一个 7TB 的表格语料库

要在这个规模上预训练，得先有足够大的表格数据。作者从 Kaggle 平台整理出一个约 7TB 的表格语料库，横跨 303 个领域、283K 张表、约 13B 条样本。平均每张表约有 28.7 个数值列、0.4 个类别列和 7.7 个文本列；占比最大的领域是投资（Investing，71K 张表）、时间序列分析（65K 张）、金融（52K 张）、经济（47K 张）和游戏（32K 张）。为避免数据泄漏，用于评测的 Kaggle 任务被明确排除在预训练之外。

不止分类回归：零样本、增量列与缺失值

预训练带来的迁移能力，在几个更贴近真实数据科学的场景里体现得更明显。零样本（zero-shot）设置下，模型不做任何针对目标数据的微调，也能在部分数据集上给出可用的预测，例如 IO 上 AUC 达 0.94、CA 上达 0.89，明显高于随机初始化，说明大规模预训练确实赋予了模型一定的可迁移推理能力。

另一个很实际的场景是“表会长出新列”。作者从训练集中移除 k 列、再用完整测试集推理，以此模拟增量列。得益于 TabUnit 的灵活性，UniTabE 对新增列的适应性明显更好：只移除 1 列时 AUC 仅下降 3.5%，不到依赖固定表结构的 TransTab-LSTM（7.5%）的一半。

表 2 增量列场景下的 AUC 相对下降（%，↓ 越小越好）。移除 k 列训练、用完整测试集推理，UniTabE 的性能损失最小。

方法	移除 1 列	移除 2 列
TransTab-LSTM	7.5	11.7
UniTabE（从零训练）	5.3	8.4
UniTabE（微调）	3.5	5.8

在缺失值填充上，UniTabE 也把任务统一成“在提示下生成目标”。对数值列它给出回归预测，对文本列它直接生成文本，在多个数据集上的 MAE 和 BLEU 都优于均值/众数、线性回归乃至 GPT-3 的对照，显示这套“提示 + 生成”的框架能自然覆盖异构的填充需求。

消融：每个组件都在出力

消融实验确认了架构里每一块都不是摆设。把连接列名与取值的 Linking Layer 去掉，平均 AUC 从 0.83 掉到 0.75，是单项影响最大的；去掉注入数据类型的 Fuse Layer 也会下降；两者都去掉更糟。两个预训练目标——多单元格掩码与对比学习——各去掉其一都会掉分，说明它们互为补充。一个有意思的发现是：单层解码器在下游 AUC 上反而优于 3 层、6 层解码器，这正好支持“把解码器做浅、让知识留在编码器”的设计取舍（更深的解码器虽然生成 BLEU 更高，却把可复用知识分走了）。

表 3 公开基准上的消融结果（平均 AUC，↑ 越高越好）。去掉 Linking Layer 掉分最多，两个预训练目标也都有贡献。

配置	平均 AUC
UniTabE（完整）	0.83
去掉 Fuse Layer	0.77
去掉 Linking Layer	0.75
同时去掉 Fuse 与 Linking	0.72
去掉多单元格掩码（MCM）	0.81
去掉对比学习（CL）	0.79

小结：把 NLP 的预训练范式带进结构化表格

UniTabE 想传达的核心信息是：重塑了语言与视觉的预训练范式，同样可以延伸到结构化的表格数据，关键在于尊重表格的结构，而不是把它拉平成文本。用列名、取值和数据类型忠实地表示每个单元格，用 Transformer 精炼整张表，再用自由文本提示适配新任务，在数十亿条样本上预训练之后，就得到一个能跨任务迁移、能超过 XGBoost、还能优雅处理缺失值和增量列的通用表格模型。

当然，这项工作也有它清晰的边界：结论建立在从 Kaggle 采集的特定语料（以数值和文本列为主、类别列很少）之上，评测集中在分类、回归、填充、零样本与增量列等任务；把它推广到更广的表格应用，仍需要更多验证。但作为“表格基础模型”这条路上的一个扎实进展，UniTabE 给出的答案是明确的：只要特征处理尊重单元格级结构，预训练在表格世界里同样成立。